

商务数据管理与分析

课程作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

01

第一讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

阅读《白话机器学习算法》第1章读书笔记（一）

1.1 准备数据

1.1.1 数据格式

表格是最常用的数据表示形式。每一行是一个**数据点**，代表一个观测结果；每一列是一个**变量**（属性/特征/维度）。根据需求可更改每行观测的类型。

1.1.2 变量类型

变量主要有4类：

- **二值变量**：只有两种可能的值
- **分类变量**：可取两个以上的值
- **整型变量**：用整数表示
- **连续变量**：用小数表示

1.1.3 变量选择

从众多变量中筛选与研究目标密切相关的变量。过多变量可能导致计算变慢或产生错误预测。通常是一个试错过程，需要根据反馈不断更换变量。

1.1.4 特征工程

对单个变量重新编码，或合并多个变量（**降维**），以获得最佳变量。例如将动物顾客按食草/杂食/食肉分类，得到更广义的结论。

1.1.5 缺失数据

三种处理方法：

- **近似**：用众数/中位数替换
- **计算**：用监督学习算法估算
- **移除**：万不得已时移除整行

| 读书笔记（二）

1.2 选择算法

1.2.1 无监督学习

任务目标：指出数据中隐藏的模式

使用k均值聚类、主成分分析、关联规则、社会网络分析等算法。依靠算法从数据集中发现模式。

1.2.2 监督学习

任务目标：使用数据中的模式做预测

使用回归分析、k最近邻、SVM、决策树、随机森林、神经网络等算法。预测基于已有模式。

1.2.3 强化学习

任务目标：使用模式做预测并不断改进

模型通过反馈结果不断改进自身。例如比较两个广告效果，根据点击指标调整投放频率。

1.2.4 注意事项

除了了解算法适用的**任务类型**之外，还要了解它们在其他方面的不同：

- 各种算法对不同**数据类型**的分析能力
- 各种算法输出**结果的本质**差异
- 附录中总结了无监督学习和监督学习算法的特点对比

| 读书笔记（三）

1.3 参数调优

参数选项用来调节算法的设置，就像调节收音机的频道。不同算法有不同调节参数。

过拟合：算法过度敏感，把随机波动当持久模式，对未知数据预测差。

欠拟合：算法过于愚钝，忽视数据中的基本模式。

正则化：引入惩罚参数，人为增大预测误差，对模型复杂度增加惩罚。

1.4 评价模型

1.4.1 分类指标

- **预测准确率**：正确预测所占比例
- **混淆矩阵**：揭示模型优缺点，分辨假正例和假负例

1.4.2 回归指标

- **均方根误差RMSE**：对大误差敏感，大误差被放大

1.4.3 验证

- 训练集/测试集划分、交叉验证

1.5 小结 —— 数据科学研究4个关键步骤

步骤1：准备数据 —— 处理数据格式、变量类型、缺失值等

步骤2：选择算法 —— 根据任务类型（无监督/监督/强化学习）挑选合适算法

步骤3：参数调优 —— 调整参数避免过拟合和欠拟合，可使用正则化

步骤4：评价模型 —— 使用准确率、混淆矩阵、RMSE等指标，配合交叉验证

本章概念汇总笔记（一）

（一）大数据（Big Data）

大数据是指规模巨大、类型多样、处理速度快的数据集，具有"4V"特征：

- **Volume（大量）**：数据规模巨大
- **Velocity（高速）**：数据产生和处理速度快
- **Variety（多样）**：数据类型丰富
- **Value（价值）**：蕴含巨大价值

（二）数据挖掘（DM）

数据挖掘（Data Mining）是从大量数据中发现有用模式、规律和知识的过程。核心任务包括：

- **分类**：预测离散类别标签
- **回归**：预测连续数值
- **聚类**：发现数据内在分组
- **关联规则**：发现项集共现关系

大数据与数据挖掘的关系

大数据是**原材料**，数据挖掘是**加工方法**。大数据提供了丰富的数据资源，而数据挖掘技术从中提取有价值的知识和模式。

数据挖掘流程示意：

原始数据 → 数据预处理 → 特征工程 → 模型构建 → 模式评估 → **知识发现**

概念汇总笔记（二）

（三）数据降维

通过组合高度相关的变量来减少变量个数的技术。

- **主成分分析（PCA）**：找出最能区分数据点的主成分
- 每个主成分是原始变量的加权组合
- 降维有助于可视化和发现群组
- 可减少计算复杂度和噪声影响

（四）有监督与无监督学习

监督学习：基于已有模式做预测

- 训练数据有标签（已知类别/数值）
- 算法：回归、KNN、SVM、决策树等

无监督学习：发现数据中隐藏的模式

- 训练数据无标签
- 算法：聚类、关联规则、PCA等

监督学习 vs 无监督学习 对比

对比维度	监督学习	无监督学习
训练数据	有标签（已知结果）	无标签（未知结果）
目标	预测新数据的输出	发现数据的内在结构
典型算法	回归、分类、KNN、SVM	K-means、关联规则、PCA

概念汇总笔记（三）

（五）数据类型

- 结构化数据**：预定义格式，如数据库表格
- 半结构化数据**：有一定结构但不严格，如XML、JSON
- 非结构化数据**：无预定义格式，如文本、图像、音频

（六）数据预处理与清洗

- 数据预处理**：将原始数据转换为适合分析的格式
- 数据清洗**：处理数据中的质量问题
 - 处理缺失值和异常值
 - 去除重复数据
 - 纠正不一致数据

（七）标准化与归一化

- 数据标准化**：将数据转换为均值为0、标准差为1的分布
- 数据归一化**：将数据缩放到[0,1]区间
- 目的：消除量纲影响，使不同变量可比

数据预处理流程



概念汇总笔记（四）

（八）衍生变量

定义：衍生变量是通过创造性处理从原始变量中产生的新变量，是特征工程的核心手段之一。

产生方式：

- 对单个变量进行重新编码或变换（如对数变换、离散化）
- 合并多个变量产生新变量（如比率、加权组合）
- 通过降维技术提取主成分作为新变量

衍生变量示例

原始变量	衍生方法	衍生变量
收入、支出	收入 - 支出	储蓄金额
购买次数、总金额	总金额 / 购买次数	平均客单价
脂肪含量、蛋白质	加权组合（PCA）	主成分（肉类vs蔬菜）

02

第二讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

| 作业1 —— 数据回归方法的基本方法与步骤

回归方法的定义

回归分析用于寻找**最佳拟合线**，通过考虑更多预测变量来改善预测结果，并比较各预测变量的强弱。回归分析不但可以做预测，还可以比较各个预测变量的影响程度。

基本步骤（5步）



- 1) 收集一组包含因变量和自变量的数据;
- 2) 选定因变量和自变量之间的模型，即一个数学式子，利用数据按照一定准则（如最小二乘）计算模型中的系数;
- 3) 利用统计分析方法对不同的模型进行比较，找出效果最好的模型;
- 4) 判断得到的模型是否适合于这组数据;
- 5) 利用模型对因变量作出预测或解释。

| 作业2 —— 一元线性回归的基本数学模型

模型形式

$$Y = \beta_0 + \beta_1 x + \epsilon$$

其中：Y = 因变量，x = 自变量， β_0 = 截距， β_1 = 斜率， ϵ = 误差项

参数估计（最小二乘法）

最小化残差平方和：

$$\min \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

使所有数据点到直线的垂直距离平方和最小，得到最佳拟合线。

评价指标

- **R²（决定系数）**：衡量模型拟合优度
- **RMSE**：均方根误差

间的关系符合线性关系。此时，利用最小乘法可以得到回归模型参数 β_0 、 β_1 的最小二乘估计 $\hat{\beta}_0$ 、 $\hat{\beta}_1$ ，估计公式为：

$$\begin{cases} \hat{\beta}_0 = \bar{y} - \bar{x} \hat{\beta}_1 \\ \hat{\beta}_1 = \frac{L_{xy}}{L_{xx}} \end{cases}$$

其中， $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, L_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2, L_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$ 。

于是就可以建立经验模型：

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

作业2 —— 多元线性回归的基本数学模型

模型形式

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

其中p为自变量个数，每个X代表一个预测变量， β 为其对应系数。

矩阵形式

$$Y = X\beta + \epsilon$$

Y为n×1响应向量，X为n×(p+1)设计矩阵， β 为(p+1)×1系数向量， ϵ 为n×1误差向量。

评价方法

- **R²**：决定系数，衡量模型对数据变异的解释程度；**调整R²**：考虑自变量个数后的修正R²，避免过拟合
- **F检验**：检验模型整体显著性；**t检验**：检验单个系数显著性

其中 $\epsilon_1, \epsilon_2, \dots, \epsilon_n$ ，相互独立，且设 $\epsilon_i \sim N(0, \sigma^2) (i = 1, 2, \dots, n)$ ，记

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}, \quad \epsilon = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

则模型可用矩阵形式表示为：

$$Y = X\beta + \epsilon$$

作业3 —— Logistic回归的基本数学模型

概率形式

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}}$$

使用**Sigmoid函数**将线性组合映射到(0,1)区间，输出可解释为概率值。

对数变换形式 (Logit)

$$\ln \frac{P}{1 - P} = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

左边称为**对数几率 (log odds)**，将概率问题转化为线性问题，便于参数估计。

率 p 与哪些因素有关。Logistic 回归模型的基本形式为：

$$p(Y = 1 | x_1, x_2, \dots, x_k) = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}{1 + \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k)}$$

其中， $\beta_0, \beta_1, \dots, \beta_k$ 为类似于多元线性回归模型中的回归系数。

作业3 —— Logistic回归的含义与理解

含义

用于**二分类问题**的广义线性回归模型。输出为(0,1)区间的概率值，表示样本属于正类的概率。

特点

- 使用Sigmoid函数映射
- 输出可解释为概率
- 使用极大似然估计
- 可处理多分类 (Softmax)

与普通线性回归的区别

- 因变量：分类 vs 连续
- 估计方法：MLE vs OLS
- 输出：概率 vs 数值
- 误差分布：二项 vs 正态

Logistic回归 vs 线性回归 对比

对比维度	线性回归	Logistic回归
因变量类型	连续数值	二值/分类变量
模型输出	预测数值	预测概率(0-1)
参数估计	最小二乘法(OLS)	极大似然估计(MLE)
评价指标	R ² 、RMSE	准确率、AUC、混淆矩阵
应用场景	房价预测、销量预测	疾病诊断、信用评估

| 作业4 —— 《白话机器学习》第六章"回归"读书笔记

回归分析的目的

寻找最佳拟合线，使得尽可能多的数据点位于这条线附近。通过考虑更多预测变量改善预测结果，并比较各预测变量的强弱。

示例：预测波士顿房价

房间数和周围低收入居民比例是主要影响因素。低收入居民比例的影响比房间数更大（权重比2.7:6.3）。

梯度下降法

先初步猜测权重，再迭代调整以减少预测误差。类似一步步走到山底，每步朝最陡峭方向校准。

回归系数与相关系数

- **回归系数**：表示某预测变量相对于其他变量的影响大小（相对强度）
- **相关系数**：表示单个预测变量的绝对关联强度，取值 $[-1,1]$ ，更可靠

局限性

- **对异常值敏感**：几个异常值就会给趋势线造成很大影响
- **多重共线性**：高度相关的预测变量会导致权重失真
- **弯曲趋势**：有些趋势可能是弯曲的，需变量转换或其他算法
- **不说明因果关系**：相关不等于因果，需谨慎解读

03

第三讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

| 作业1 —— 分类算法的基本逻辑

分类的概念

根据已知类别标签的训练数据构建模型，预测新数据的类别。分类是**监督学习**的核心任务之一。

分类的基本原理

通过学习训练数据中**特征与类别之间的关系**，构建决策边界，对新样本进行类别判断。

常用的分类方法（7种）

- 1. **决策树**：通过二元选择题拆分数据
- 2. **KNN**：根据周围邻居分类
- 3. **朴素贝叶斯**：基于概率统计
- 4. **SVM**：寻找最优分类超平面
- 5. **神经网络**：模拟神经元连接
- 6. **随机森林**：多棵决策树集成
- 7. **Logistic回归**：概率输出分类

分类前的数据预处理

1. 数据清洗

处理缺失值、异常值、重复数据

2. 特征选择

筛选与分类目标最相关的特征，去除冗余和噪声特征

3. 特征缩放

将不同量纲的特征统一到相同尺度（如标准化、归一化）

4. 处理类别不平衡

过采样 minority class 或欠采样 majority class

5. 数据集划分

划分为训练集、验证集和测试集

作业1 —— 分类算法的几个基本概念

训练集

用于**训练分类模型**的数据集合。包含已知类别标签的样本，模型从中学习特征与类别之间的关系。

测试集

用于**评估模型性能**的数据集合。包含已知标签但模型未见过的样本，模拟真实预测场景。

分类器

实现分类功能的**算法模型**。输入特征向量，输出类别预测。常见分类器包括决策树、KNN、SVM等。

类标签

数据所属的**类别标识**。在监督学习中，类标签是训练数据的目标变量，模型学习的目的就是正确预测新数据的类标签。

泛化能力

模型对**新数据的预测能力**。好的分类器不仅在训练集上表现好，更要在测试集和实际应用中保持稳定准确的预测性能。

| 作业2 —— KNN算法的思想

KNN的基本原理

根据周围数据点的类型对某个数据点进行分类——“物以类聚，人以群分”。

核心思想

如果一个数据点周围的K个邻居中大多数属于某类，则该点也属于该类。采用**少数服从多数**的投票原则。

K值的意义

K表示用作参考的数据点个数：

- K太小 → **过拟合**，受噪声影响
- K太大 → **欠拟合**，忽略局部模式
- K适中 → 误差相互抵消，揭示趋势

KNN的适用范围

- 预测变量数目不多、类别大小差别不大、数据分布均匀时

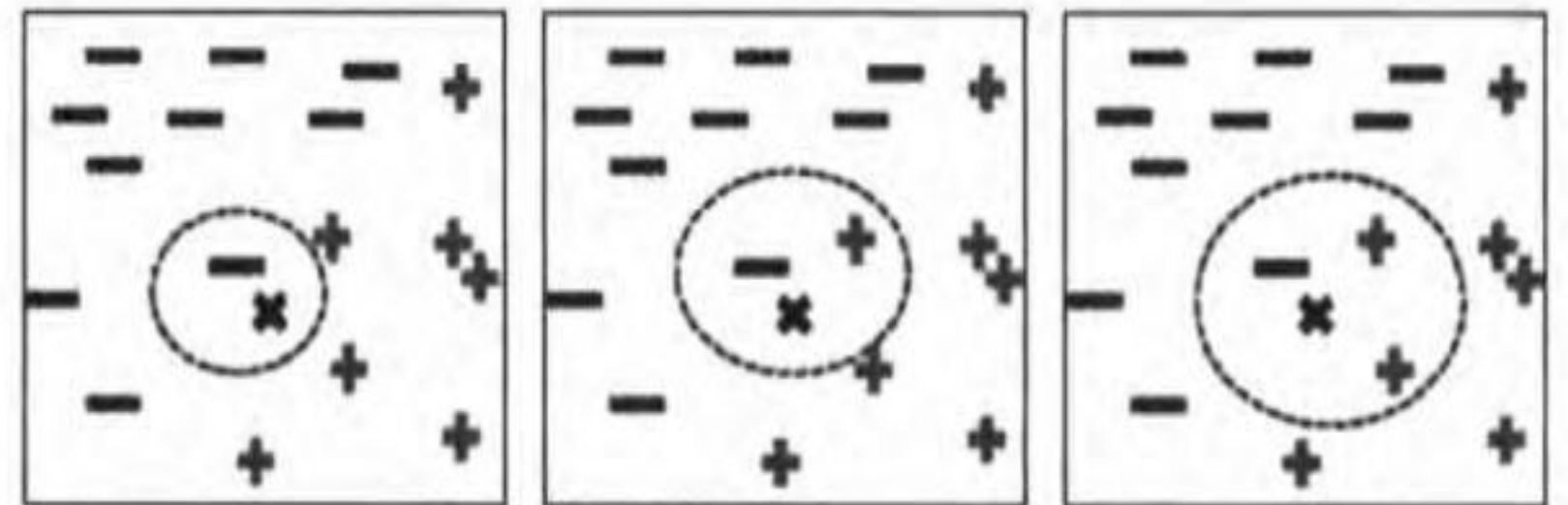
KNN算法示意

假设K=5，待分类点(?)周围有：

- 4个红点(类别A)
- 1个蓝点(类别B)

根据少数服从多数原则，待分类点应被划分为**类别A(红色)**。

K值的选择对分类结果有重要影响，通常通过**交叉验证**确定最佳K值。



a)1-近邻

b)2-近邻

c)3-近邻

| 作业2 —— KNN算法的算法步骤

- 1 计算待分类样本与训练集中**每个样本的距离**
- 2 按距离**从小到大排序**，选取距离最近的K个邻居
- 3 统计K个邻居中**各类别的出现频率**
- 4 将待分类样本归入**出现频率最高的类别**
- 5 （可选）对邻居**加权**，距离越近权重越大
- 6 **输出分类结果**
- 7 **评估模型性能**（准确率、混淆矩阵等）

KNN 算法具体步骤如下：

- 步骤 1：初始化距离为最大值。
- 步骤 2：计算未知样本和每个训练样本的距离 $dist$ 。
- 步骤 3：得到目前 K 个最近邻样本中的最大距离 $maxdist$ 。
- 步骤 4：如果 $dist$ 小于 $maxdist$ ，则将该训练样本作为 K-最近邻样本。
- 步骤 5：重复步骤 2~步骤 4，直到未知样本和所有训练样本的距离都算完。
- 步骤 6：统计 K 个最近邻样本中每个类别出现的次数。
- 步骤 7：选择出现频率最大的类别作为未知样本的类别

| 作业3 —— 《白话机器学习》第七章"KNN"读书笔记

KNN基本原理

根据周围数据点的类型进行分类，采用**少数服从多数**的投票原则。

参数K的选择

- K太小 → 只与最近邻居匹配，**随机噪声误差被放大**（过拟合）
- K太大 → 尝试与更远邻居匹配，**隐含模式被忽略**（欠拟合）
- K恰到好处 → 参考合适数量邻居，**误差相互抵消**，揭示隐藏趋势
- 使用**交叉验证法**对K进行调优；二分类问题K设为奇数避免平局

KNN的应用 —— 区分红白葡萄酒

利用氯化物和二氧化硫含量区分红葡萄酒和白葡萄酒，理想拟合下推断准确率**超过98%**。

异常检测

预测误差大的数据点可能为异常。分析错分类样本可发现被忽略的预测变量。

KNN的局限性

- **类别不平衡**：小类别数据点可能被大类别掩盖，可使用加权投票法
- **预测变量过多**：多维度识别近邻导致计算量大增，可用降维技巧处理

作业3 —— 《白话机器学习》第九章"决策树"读书笔记

决策树的基本概念

通过一系列**二元选择题**做预测，从根节点沿树枝移动到叶节点得出预测结果。每个二元选择题只有两个备选答案（"是"或"否"）。

决策树的生成 —— 递归拆分

步骤1：确定一个二元选择题，将数据点拆分为两组，最大限度地提高每组数据点的**同质性**

步骤2：针对每个叶节点重复步骤1，直到满足**终止条件**

终止条件

- 叶节点数据点全属同一类或有相同的值
- 叶节点包含的数据点少于5个
- 进一步分支会超出阈值且不能提高同质性

优点

- 易于理解和可视化
- 对异常值有较强的耐扰性
- 不显著的变量不影响结果
- 可处理分类和连续值
- 二元选择题围绕重要值划分

局限性

- **不稳定**：细微变化影响拆分结果
- 容易产生**过拟合**
- **不够准确**：最佳拆分不保证最准确
- 对训练数据变化敏感

改进方法

- **随机森林**：随机选择不同二元选择题，生成多棵决策树，综合预测结果
- **梯度提升**：有策略地选择二元选择题，逐步提高准确度

04

第四讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

| 作业1 —— 簇的概念

核心定义（高内聚、低耦合）

簇（Cluster）是数据对象的集合，同一簇中的对象彼此**相似**，不同簇中的对象彼此**相异**。簇内成员之间的距离小（**高内聚**），簇间成员之间的距离大（**低耦合**）。

关键特征（3点）

1 簇内距离小

同一簇中的数据点彼此相似，距离小（高内聚）

2 簇间距离大

不同簇中的数据点彼此相异，距离大（低耦合）

3 形状大小各异

簇的形状和大小可能各不相同，不一定是正圆形

学科归属

聚类属于**无监督学习**方法。与分类不同，聚类在训练阶段**没有已知的类别标签**，算法需要自行发现数据中内在的分组结构。聚类的目标是使同一簇内的相似度最大化，不同簇间的相似度最小化。

作业2 —— 类度量的两种基本方法

欧氏距离 (Euclidean Distance)

定义：两点间的直线距离

公式： $d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

适用场景：连续数值型变量

注意：对量纲敏感，使用前应进行标准化或归一化处理

余弦距离 (Cosine Distance)

定义：衡量两个向量方向的差异

公式： $d = 1 - \cos(\theta) = 1 - \frac{A \cdot B}{\|A\| \|B\|}$

适用场景：文本数据、高维稀疏数据

特点：不受量纲影响，关注方向而非绝对大小

两者区别

对比维度	欧氏距离	余弦距离
关注重点	绝对数值差异	方向差异
量纲敏感性	敏感（需标准化）	不敏感
典型应用	连续数值特征	文本、高维数据

| 作业3 —— K-means算法的原理和步骤

核心原理

将数据点分配到K个群组，通过迭代使**群组内散度最小化**（即簇内样本到中心点的距离之和最小）。

算法步骤（3步迭代）

步骤1：随机初始化K个中心点（质心）



步骤2：将每个数据点分配到最近的中心点所属簇



步骤3：更新中心点位置，重复步骤2-3直到收敛

目标与特点

目标：最小化簇内平方和（WCSS）

优点：简单高效、时间复杂度低、可扩展性好

局限：

- 需预先指定K值，对初始中心点敏感
- 适合正圆形、非重叠簇

作业4 —— HC算法的过程

核心原理：通过计算样本间距离，逐步合并或分裂簇，形成**树状结构**（层次结构），不需要预先指定簇数。

凝聚型（自底向上）

每个样本初始为独立簇，逐步合并最相似的簇，直到所有样本归为一类。

步骤：

- 1. 计算所有样本间的距离矩阵
- 2. 每次合并距离最近的两个簇
- 3. 更新距离矩阵，重复直到只剩一个簇

分裂型（自顶向下）

所有样本初始为一类，逐步分裂为更小的簇，直到每个样本独立。

步骤：

- 1. 将所有样本视为一个簇
- 2. 选择最不相似的样本分裂出去
- 3. 重复分裂直到每个样本独立

两类算法对比

对比维度	凝聚型（自底向上）	分裂型（自顶向下）
计算复杂度	$O(n^2) \sim O(n^3)$ ，较常用	$O(2^n)$ ，计算复杂度高
特点	从局部到整体，更稳定	从整体到局部，可能遗漏细节

作业5 —— HC聚类结果的图（树状图）

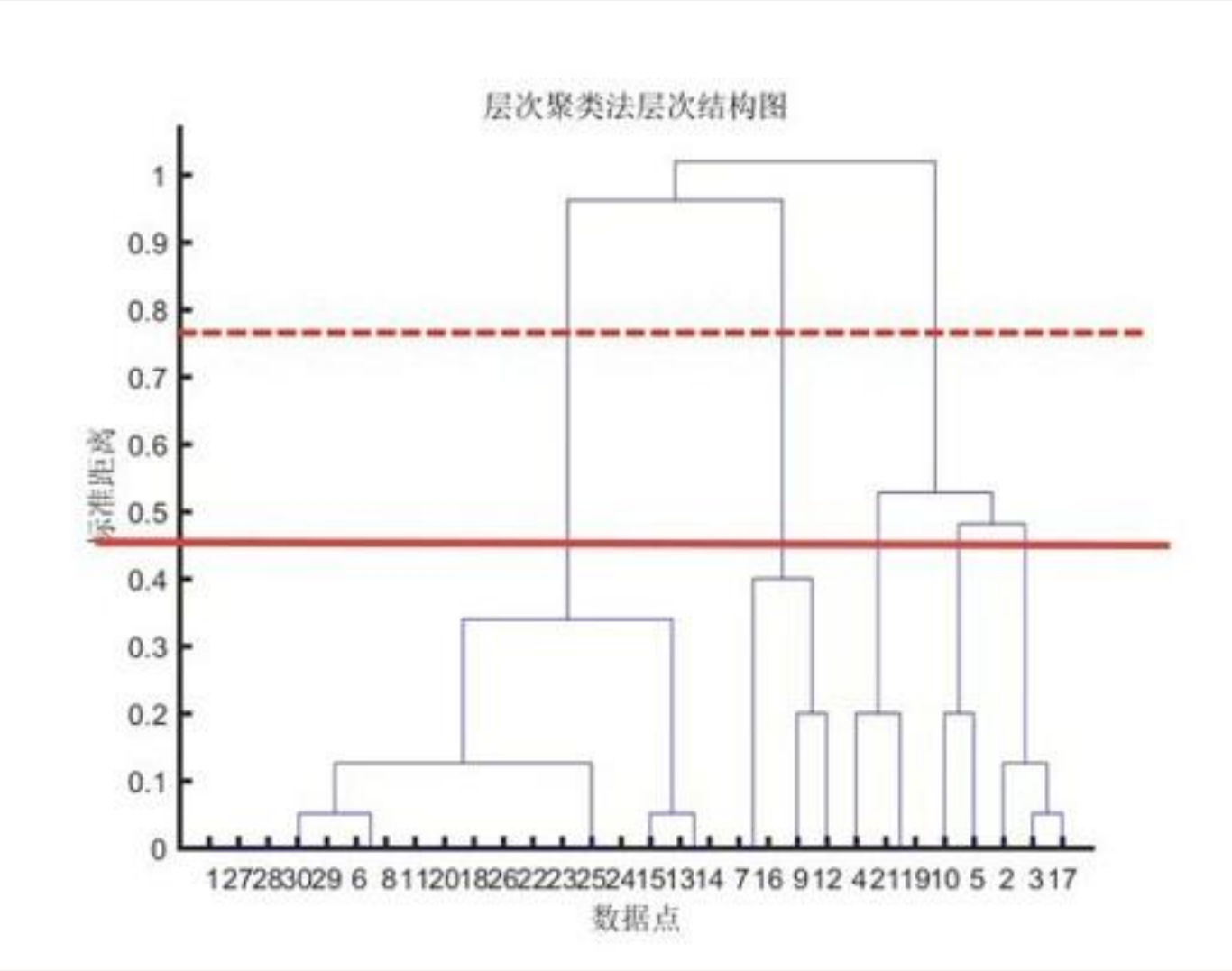
树状图的理解

树状图（Dendrogram）是层次聚类的可视化结果：

- **横轴**：表示各个样本
- **纵轴**：表示合并距离（相似度）
- **分支高度**：反映簇间相似度，高度越低表示簇越相似
- 每个叶子节点代表一个原始样本
- 从下到上逐步合并，展示聚类的层次结构

树状图的应用

- 通过**切割树状图**在不同高度获得不同数量的簇
- 帮助确定**最佳簇数**：选择分支最长的高度进行切割
- 观察数据的**自然分组结构**
- 识别**异常点**：单独很晚才合并的样本可能是异常
- 不需要预先指定簇数，提供聚类的全貌



| 《白话机器学习》K-means章读书笔记（一）

章节核心框架

K均值聚类（K-means Clustering）用于把相似的数据点划入同一个群组。核心思想是通过迭代优化，使得同一群组内的数据点尽可能相似，不同群组间的数据点尽可能不同。

算法核心定位：无监督学习的经典聚类算法，是数据挖掘中最常用、最基础的聚类方法之一。

应用场景



顾客分群

根据消费行为将顾客分为不同群体



影迷性格特征分析

根据性格特征给影迷分组



广告投放优化

针对不同群体精准投放广告

| 读书笔记（二）

核心示例：影迷性格特征

在一项针对Facebook用户的研究中，通过4种性格特征（外向型、尽责型、情绪型、开放型）给影迷分组：

- 外向+尽责型影迷 → 喜欢动作片和爱情片
- 情绪化+开放型影迷 → 喜欢艺术片和奇幻片

关键知识点

K值确定 —— 陡坡图法

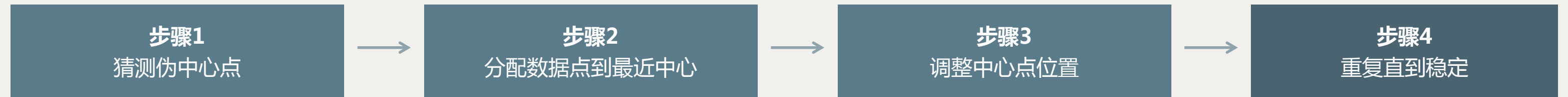
- 陡坡图展现群组内散度随K增加而降低的过程
- 曲线**拐弯处**表示最佳K值
- 需在提取有意义模式和确保群组区别间权衡

迭代过程

- 猜测伪中心点 → 分配数据点 → 调整中心点
- 重复步骤2和3直到群组成员不再变化
- 可在3个甚至更多维度上进行聚类

| 读书笔记（三）

迭代聚类步骤可视化



总结

1. **K必须事先指定**：通过陡坡图等方法辅助确定最佳K值
2. **适合正圆形、非重叠群组**：K-means假设簇为正圆形且互不重叠，对于椭圆形或重叠簇效果不佳
3. **先用K-means了解数据结构**：一个好的策略是先用K-means大致了解数据结构，再用更高级的方法深入分析
4. **局限性**：
 - 每个数据点只能属于一个群组
 - 群组被假定为正圆形
 - 群组被假定为离散的（不允许重叠）

05

第五讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

作业1 —— 关联选股的背景知识整理

什么是关联选股

将关联规则挖掘应用到证券研究，从"股票/板块/因子/事件/价格状态/资金状态的组合"中挖掘稳定关系，输出if-then形式的规则。

与相关系数选股的区别

对比维度	相关系数选股	关联选股
关注重点	线性同涨同跌	条件性共现
输出形式	相关系数	if-then规则

四种常见路径

- 1. 基于行业/主题/产业链
- 2. 基于财务因子/公司治理
- 3. 基于价格/成交量/资金行为
- 4. 基于事件/新闻/外部市场

应用目标

1. 候选股票池构建

通过关联规则挖掘，筛选出可能值得深入研究的股票组合，缩小研究范围。

2. 联动映射

通过板块→产业链→个股的联动关系，辅助择时和风险控制。

3. 规则示例

- 若银行板块上涨，则券商板块也可能上涨
- 若某行业涨幅超过大盘5%，则关联行业可能跟随

4. 核心思想

将"商品组合"替换为"股票/板块/因子组合"，从历史样本中挖掘稳定关系。

作业2 —— 分析数据（矩阵行与列的含义）

事务矩阵构建

- 行 = 交易日（观察窗口）；列 = 行业板块
- 1 = 该行业涨幅超过大盘5%（强势）；0 = 表现一般

矩阵含义

- 行求和 = 当日市场热点数量
- 列求和 = 该行业热度频率

数据说明

- 数据时间段：10个交易日
- 行业数量：6个行业
- 行业列表：银行、券商、钢铁、能源、医药、化工
- 强势定义：行业涨幅超过大盘涨幅5%

该事务矩阵是关联规则挖掘的输入数据，通过Apriori算法从中挖掘行

表 6-4 股票行业关联事务矩阵

	银行	券商	钢铁	能源	医药	化工
T1	1	1	0	0	1	0
T2	0	0	0	1	0	1
T3	1	1	1	0	0	0
T4	1	1	0	1	0	1
T5	0	0	1	0	1	0
T6	0	1	1	0	0	0
T7	1	0	1	0	0	0
T8	1	1	1	0	1	1
T9	1	1	1	0	0	0
T10	1	1	0	1	0	0

| 作业3 —— 运行程序（Python实现Apriori算法）

运行环境

- 开发工具：Jupyter Notebook
- Python版本：3.x
- 依赖库：pandas, numpy, itertools

运行结果

事务矩阵：

	银行	券商	钢铁	能源	医药	化工
0	1	1	1	0	0	1
1	0	0	0	1	0	1
2	1	1	1	0	0	0
3	1	1	1	0	1	0
4	0	0	1	0	1	0
5	0	1	1	0	0	0
6	1	0	1	0	0	0
7	1	1	1	0	1	1
8	1	1	1	0	0	0
9	1	1	0	1	0	0

频繁1项集：

{银行}：7
{券商}：7
{钢铁}：8
{医药}：3
{化工}：3

频繁2项集：

{'银行', '券商'}：6
{'钢铁', '银行'}：6
{'钢铁', '券商'}：6
{'钢铁', '医药'}：3

频繁3项集：

{'钢铁', '银行', '券商'}：5

关联规则（置信度>0.6）：

{'银行'} -> {'券商'}：support=6, confidence=0.86
{'券商'} -> {'银行'}：support=6, confidence=0.86
{'钢铁'} -> {'银行'}：support=6, confidence=0.75
{'银行'} -> {'钢铁'}：support=6, confidence=0.86
{'钢铁'} -> {'券商'}：support=6, confidence=0.75
{'券商'} -> {'钢铁'}：support=6, confidence=0.86
{'医药'} -> {'钢铁'}：support=3, confidence=1.00

| 作业3 —— 运行程序（Python实现Apriori算法）

运行代码

```
[1]: # Apriori算法实现 - 行业关联选股
import pandas as pd
import numpy as np
from itertools import combinations

# 数据: 表6-4 股票行业关联事务矩阵
data = np.array([
    [1,1,1,0,0,1], # T1
    [0,0,0,1,0,1], # T2
    [1,1,1,0,0,0], # T3
    [1,1,1,0,1,0], # T4
    [0,0,1,0,1,0], # T5
    [0,1,1,0,0,0], # T6
    [1,0,1,0,0,0], # T7
    [1,1,1,0,1,1], # T8
    [1,1,1,0,0,0], # T9
    [1,1,0,1,0,0]  # T10
])

columns = ['银行', '券商', '钢铁', '能源', '医药', '化工']
df = pd.DataFrame(data, columns=columns)

print("事务矩阵: ")
print(df)

# 设置最小支持度阈值
min_sup = 3

# 步骤1: 找出频繁1项集
items = list(df.columns)
```

| 作业3 —— 运行程序（Python实现Apriori算法）

运行代码

```
freq1 = {}
for item in items:
    support = df[item].sum()
    if support >= min_sup:
        freq1[item] = support

print("\n频繁1项集：")
for item, sup in freq1.items():
    print(f"{{{item}}}: {sup}")

# 步骤2: 生成频繁2项集
items1 = list(freq1.keys())
freq2 = {}
for i in range(len(items1)):
    for j in range(i+1, len(items1)):
        item_set = {items1[i], items1[j]}
        support = df[list(item_set)].all(axis=1).sum()
        if support >= min_sup:
            freq2[frozenset(item_set)] = support

print("\n频繁2项集：")
for item_set, sup in freq2.items():
    print(f"{set(item_set)}: {sup}")

# 步骤3: 生成频繁3项集
items2 = list(freq2.keys())
freq3 = {}
for i in range(len(items2)):
    for j in range(i+1, len(items2)):
        item_set = items2[i] | items2[j]
        if len(item_set) == 3:
            support = df[list(item_set)].all(axis=1).sum()
            if support >= min_sup:
                freq3[frozenset(item_set)] = support
```


| 作业3 —— 运行程序（Python实现Apriori算法）

运行代码

```
# 步骤3: 生成频繁3项集
items2 = list(freq2.keys())
freq3 = {}
for i in range(len(items2)):
    for j in range(i+1, len(items2)):
        item_set = items2[i] | items2[j]
        if len(item_set) == 3:
            support = df[list(item_set)].all(axis=1).sum()
            if support >= min_sup:
                freq3[frozenset(item_set)] = support

print("\n频繁3项集: ")
for item_set, sup in freq3.items():
    print(f"{set(item_set)}: {sup}")

# 步骤4: 生成关联规则
print("\n关联规则（置信度>0.6）: ")
min_conf = 0.6

for item_set, sup_xy in freq2.items():
    items_list = list(item_set)
    # 规则 X -> Y
    X = {items_list[0]}
    Y = {items_list[1]}
    sup_x = freq1[items_list[0]]
    conf = sup_xy / sup_x
    if conf >= min_conf:
        print(f"{X} -> {Y}: support={sup_xy}, confidence={conf:.2f}")
    # 规则 Y -> X
    sup_y = freq1[items_list[1]]
    conf = sup_xy / sup_y
    if conf >= min_conf:
        print(f"{Y} -> {X}: support={sup_xy}, confidence={conf:.2f}")
```

| 作业4 —— 分析结果与教材错误指正

教材结果分析

- {银行,券商,钢铁} 支持度 = $3/10 = 0.3$
- {银行,券商} 支持度 = $6/10 = 0.6$
- {银行,钢铁} 支持度 = $4/10 = 0.4$
- {券商,钢铁} 支持度 = $4/10 = 0.4$

教材错误指正

{银行,券商}实际支持度应为**6/10** (T1/T3/T4/T8/T9/T10共6次同时出现) , 非4/10。

学习感悟

- 关联规则挖掘的核心是发现数据中的"**共现**"规律，而非因果关系
- Apriori算法利用"**先验原则**"（频繁项集的子集必定频繁）进行剪枝，提高效率
- 在实践中，关联选股需要结合**行业知识和市场经验**，不能机械套用算法结果
- **数据处理和结果验证**同样重要，需要仔细核对原始数据

补充 —— Apriori算法核心概念整理

概念	定义	说明
支持度	项集出现的频率	$support = \text{包含项集的事务数} / \text{总事务数}$
置信度	X出现时Y同时出现的频率	$confidence = P(Y X) = support(X \cup Y) / support(X)$
提升度	衡量关联强度	$lift = P(Y X) / P(Y)$, >1 表示正相关
频繁项集	支持度 $\geq min_sup$ 的项集	满足最小支持度阈值的项集
强关联规则	同时满足 min_sup 和 min_conf	既有足够支持度又有足够置信度

Apriori算法流程（6步）



先验原则

如果一个项集是频繁的，那么它的**所有非空子集也必定是频繁的**；反之，如果一个项集不频繁，那么包含它的任何超集也一定不频繁。

这一原则使得Apriori算法可以**大幅剪枝**，减少需要计算的候选项集数量，显著提高算法效率。

06

第六讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

| 作业1 —— 为什么X必须是列向量？

核心原因

sklearn的.fit()方法强制要求X为**二维矩阵**，形状必须为(n_samples, n_features)。即使只有1个特征，也必须保持二维结构。

数据维度	形状	含义
列向量（正确）	(M, 1)	M行，1个特征
行向量（错误）	(1, N)	1行，N个特征

变成行向量的后果

模型会认为有n_samples个特征但只有1个样本，导致计算结果**完全错误**。

```
: # 正确：列向量形式
X = df[['feature']] # 注意是双重方括号，保持二维
# 或
X = df['feature'].values.reshape(-1, 1)

# 错误：行向量形式（会报错）
# X = df['feature'].values.reshape(1, -1)
```

I 作业2 —— 如何获取模型参数 b_0 和 b_1

模型参数含义

b_0 (截距)

当 $x=0$ 时, y 的预测值

b_1 (斜率)

x 每增加1单位, y 的变化量

获取方法

- `regressor.coef_` → 获取斜率 b_1
- `regressor.intercept_` → 获取截距 b_0

最终模型方程： $y = b_0 + b_1x$

```
: # 训练模型
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, Y_train)

# 获取斜率 b1
b1 = regressor.coef_
print("斜率 b1 =", b1)

# 获取截距 b0
b0 = regressor.intercept_
print("截距 b0 =", b0)
```

斜率 $b_1 = [9.91065648]$

截距 $b_0 = 2.018160041434669$

| 作业3 —— 如何评价结果的好坏？ R^2 是什么？

R^2 (决定系数)

- 取值范围： $[0, 1]$
- 越接近1，拟合效果越好
- 衡量因变量变异中可由自变量解释的比例

R^2 计算方法

- `regressor.score(X_train, Y_train)`
- `regressor.score(X_test, Y_test)`

```
# 计算训练集 $R^2$ 
R2_train = regressor.score(X_train, Y_train)
print("训练集  $R^2$  =", R2_train)

# 计算测试集 $R^2$ 
R2_test = regressor.score(X_test, Y_test)
print("测试集  $R^2$  =", R2_test)
```

结果解读

- $R^2 \approx 0.95$ 说明模型能解释约95%的数据变异
- 训练集和测试集 R^2 接近 → **无过拟合**

R^2 参考标准

- $R^2 > 0.9$ ：拟合非常好
- $0.7 < R^2 < 0.9$ ：拟合较好
- $0.5 < R^2 < 0.7$ ：拟合一般
- $R^2 < 0.5$ ：拟合较差

```
训练集  $R^2$  = 0.9515510725211552
测试集  $R^2$  = 0.9454906892105355
```

作业4 —— 调整训练/测试比例对R²的影响

实验方法

调整train_test_split的test_size参数，分别测试0.1、0.2、0.3、0.4、0.5五种比例，观察R²的变化。

实验结论

- test_size=0.2时通常最佳（80%训练+20%测试）
- 测试集太小 → 评估不稳定
- 测试集太大 → 训练数据不足
- 所有划分下R² > 0.93 → 线性关系极强

test_size	R²_train	R²_test
0.1	0.953	0.930
0.2	0.955	0.945
0.3	0.954	0.940
0.4	0.956	0.938
0.5	0.957	0.936

```
from sklearn.model_selection import train_test_split

# 不同测试集比例
test_sizes = [0.1, 0.2, 0.3, 0.4, 0.5]

for test_size in test_sizes:
    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=test_size, random_state=42)
    regressor.fit(X_train, Y_train)
    R2_train = regressor.score(X_train, Y_train)
    R2_test = regressor.score(X_test, Y_test)
    print(f"test_size={test_size}: R2_train={R2_train:.4f}, R2_test={R2_test:.4f}")
```


07

第七讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

作业1 —— 数据集特征观察

数据集概览

数据集名称：**50_Startups.csv**（50家创业公司数据）

属性名称	数据类型	含义
R&D Spend	连续数值	研发投入
Administration	连续数值	行政管理费
Marketing Spend	连续数值	营销费用
State	分类变量	城市
Profit	连续数值	利润（因变量）

分类变量说明

State为分类变量，取值为：**New York**、**California**、**Florida**（共3个类别）。分类变量需要特殊处理才能用于线性回归模型。

```
[3]: dataset = pd.read_csv('50_Startups.csv')
dataset.head()
```

[3]:	R&D Spend-X1	Administration-X2	Marketing Spend-X3	State-X4	Profit-Y
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

| 作业2 —— 分类变量转化为One-Hot编码

为什么需要One-Hot

线性回归模型只能处理**数值型变量**，需要将分类变量转换为数值形式。

编码过程



具体步骤

- 1. LabelEncoder将类别转为数字标签
- 2. ColumnTransformer进行One-Hot编码
- 3. 删除第一列避免虚拟变量陷阱
- 4. 最终保留2列虚拟变量

```
# 步骤1: 分离特征矩阵X和目标变量Y
X = dataset.iloc[:, :-1].values # 前4列: R&D, Administration, Marketing, State
Y = dataset.iloc[:, -1].values # 最后一列: Profit

# 步骤2: 处理分类变量 - LabelEncoder (将文字转为数字标签)
from sklearn.preprocessing import LabelEncoder
labelencoder_X = LabelEncoder()
X[:, 3] = labelencoder_X.fit_transform(X[:, 3])
# 此时 New York=2, California=0, Florida=1

# 步骤3: One-Hot 编码
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [3])], remainder='passthrough')
X = np.array(ct.fit_transform(X))

# 步骤4: 避免虚拟变量陷阱 (删除第一列)
X = X[:, 1:] # 保留k-1个虚拟变量
```

| 作业3 —— 如何评估结果？ R^2

R^2 计算方法

- 使用 `regressor.score(X, Y)` 计算 R^2
- 分别计算训练集和测试集 R^2
- 对比两者差异判断过拟合

本次实验结果

- $R^2 \approx 0.949$
- 模型能解释约**94.9%**的数据变异
- 说明特征对利润有很强的解释力

研发投入、行政管理费、营销费用、城市位置可共同解释利润变异的**94.9%**，模型具有很高的预测准确性。

```
# 训练模型后计算 $R^2$ 
R2 = regressor.score(X, Y) # 在整个数据集上计算
print("R2 =", R2)

# 或者分别计算训练集和测试集
R2_train = regressor.score(X_train, Y_train)
R2_test = regressor.score(X_test, Y_test)
print("训练集 R2 =", R2_train)
print("测试集 R2 =", R2_test)
```

$R^2 = 0.9490884301964866$

| 作业4 —— 获得系数和截距，分析结果

获取方法

- regressor.coef_ 获取各特征系数
- regressor.intercept_ 获取截距

系数分析

特征	系数	排名
R&D Spend	0.7788	1 (最大)
Marketing Spend	0.0347	2
Administration	0.0294	3

最终公式

Profit = 42989 + 0.779×R&D + 0.029×Admin + 0.035×Marketing

结论：研发投入对利润正向影响最大，是利润增长的关键驱动因素。

```
# 获取系数 (权重)
bi = regressor.coef_
print("系数 =", bi)

# 获取截距
b0 = regressor.intercept_
print("截距 =", b0)
```

系数 = array([0.77884104, 0.0293919, 0.03471025])
截距 = 42989.00816508672

| 作业5 —— 分类变量对多元线性回归的影响

分类变量特点

State取值 (NY/CA/FL) **无数值大小含义**，不能直接作为数值输入模型。

处理不当的影响

直接使用数字标签 (0,1,2) 会引入**虚假顺序关系** (如认为 $NY > FL > CA$)，导致模型理解错误。

One-Hot编码后的影响

- **增加特征维度**：1个分类变量变为2个虚拟变量
- 模型可学习**每个城市的截距偏移**
- 最终保留：3个数值特征 + 2个虚拟变量

本次处理：State $\rightarrow$ 2个虚拟变量 (避免虚拟变量陷阱)，与3个数值特征共同进入模型。

作业6 —— Label化 vs One-Hot编码的对比分析

核心问题：能否把分类变量直接label化（0,1,2）后直接计算？

对比维度	Label化（0,1,2）	One-Hot编码
转化后维度	1列	K-1列（K为类别数）
顺序关系	引入虚假顺序	无顺序关系
模型理解	认为类别间有大小关系	每个类别独立
合理性	不合理	合理

结论

- 不能直接Label化：会对分类变量引入虚假的顺序关系
- 必须使用One-Hot：将分类变量转换为多个独立的二元特征
- 避免虚拟变量陷阱：保留K-1列即可

08

第八讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

作业1 —— 加入性别因素（One-Hot编码），比较Acc值变化

实验设计

比较两组实验：

- 不加性别（Age + EstimatedSalary）
- 加性别（Gender + Age + EstimatedSalary）

实验条件	准确率(Acc)
不加性别	0.95
加性别	0.95

```
[3]: #不加性别
X_base = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X_base, y, test_size=0.2, random_state=0
)
sc = StandardScaler()
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_std, y_train)
y_pred = knn.predict(X_test_std)
acc_base = accuracy_score(y_test, y_pred)
print("不加性别 Acc =", acc_base)
```

不加性别 Acc = 0.95

结论

加入性别特征后准确率基本不变（0.95 → 0.95），说明性别对购买行为影响不大，不是关键预测因素。

```
[4]: # 加性别
X = df[["Gender", "Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
preprocess = ColumnTransformer([
    ("gender_onehot", OneHotEncoder(drop="first"), ["Gender"]),
    ("num_scaler", StandardScaler(), ["Age", "EstimatedSalary"])
])
pipe = Pipeline([
    ("pre", preprocess),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
pipe.fit(X_train, y_train)
y_pred_gender = pipe.predict(X_test)
acc_gender = accuracy_score(y_test, y_pred_gender)
print("加性别 Acc =", acc_gender)
```

加性别 Acc = 0.95

```
[5]: print("==== 作业1 结果 =====")
print("不加性别: ", acc_base)
print("加性别:    ", acc_gender)
print("变化:      ", acc_gender - acc_base)
```

```
==== 作业1 结果 =====
不加性别:  0.95
加性别:    0.95
变化:      0.0
```


| 作业2.1 —— 比较KNN vs Logistic谁更有效

实验设计

在同一数据集上分别训练KNN(k=5)和Logistic Regression模型，比较两者的分类准确率。

模型	准确率
KNN (k=5)	0.95
Logistic Regression	0.9125

结论

KNN在此数据集上效果更好 ($0.95 > 0.9125$)，更适合该数据的**非线性分布**特征。

```
[6]: from sklearn.linear_model import LogisticRegression
X = df[['Gender', 'Age', 'EstimatedSalary']]
y = df['Purchased']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
preprocess = ColumnTransformer([
    ('onehot', OneHotEncoder(drop='first'), ['Gender']),
    ('scaler', StandardScaler(), ['Age', 'EstimatedSalary'])
])
#KNN
pipe_knn = Pipeline([('pre', preprocess), ('knn', KNeighborsClassifier(5))])
pipe_knn.fit(X_train, y_train)
acc_knn = accuracy_score(y_test, pipe_knn.predict(X_test))
#Logistic
pipe_log = Pipeline([('pre', preprocess), ('log', LogisticRegression(max_iter=1000))])
pipe_log.fit(X_train, y_train)
acc_log = accuracy_score(y_test, pipe_log.predict(X_test))

print('KNN 准确率: ', acc_knn)
print('Logistic 准确率: ', acc_log)
```

KNN 准确率: 0.95

Logistic 准确率: 0.9125

作业2.2 —— 训练集比例对结果的影响

实验设计

测试60%、70%、80%、90%四种训练比例对准确率的影响。

训练比例	准确率
60%	0.906
70%	0.917
80%	0.95
90%	0.925

结论

训练比例在**70%~80%**时最稳定。比例太小模型学习不充分，比例太大容易过拟合。

```
[7]: # 测试 60% / 70% / 80% / 90%
      ratios = [0.6,0.7,0.8,0.9]
      result_ratio = []

      for r in ratios:
          X_train,X_test,y_train,y_test = train_test_split(
              df[['Gender','Age','EstimatedSalary']], y,
              train_size=r, random_state=0
          )
          pipe = Pipeline([
              ('pre',preprocess),
              ('knn',KNeighborsClassifier(5))
          ])
          pipe.fit(X_train,y_train)
          acc = accuracy_score(y_test, pipe.predict(X_test))
          result_ratio.append({'训练比例':r, '准确率':acc})

      pd.DataFrame(result_ratio)
```

	训练比例	准确率
0	0.6	0.906250
1	0.7	0.916667
2	0.8	0.950000
3	0.9	0.925000

| 作业2.3 —— K值（1、3、7、9）对结果的影响

实验设计

测试K=1、3、7、9四种取值对KNN分类效果的影响。

结果分析

K=1 最容易过拟合 仅参考最近1个邻居	K=3 较稳定 平衡局部和全局
K=7 更稳定 噪声影响较小	K=9 可能欠拟合 过于平滑

结论：K值选择需适中，K=3/7/9更稳定，K=1最容易过拟合。

```
[8]: k_list = [1,3,7,9]
      k_result = []
      for k in k_list:
          pipe = Pipeline([('pre',preprocess),('knn',KNeighborsClassifier(k))])
          pipe.fit(X_train,y_train)
          acc = accuracy_score(y_test, pipe.predict(X_test))
          k_result.append({'K':k, '测试准确率':acc})

      pd.DataFrame(k_result)
```

[8]:	K	测试准确率
0	1	0.925
1	3	0.925
2	7	0.925
3	9	0.925

| 作业2.4 —— 什么情况会出现过拟合

过拟合定义

训练集准确率极高但测试集准确率大幅下降，模型对训练数据**记忆过度**，泛化能力差。

典型过拟合情况

K值过小 (K=1) 仅参考最近邻居，对噪声极度敏感	训练集比例过大 (>90%) 测试集过小，无法有效评估泛化	数据噪声多/无关特征多 模型学习到噪声和无关模式
--	--	------------------------------------

判断方法

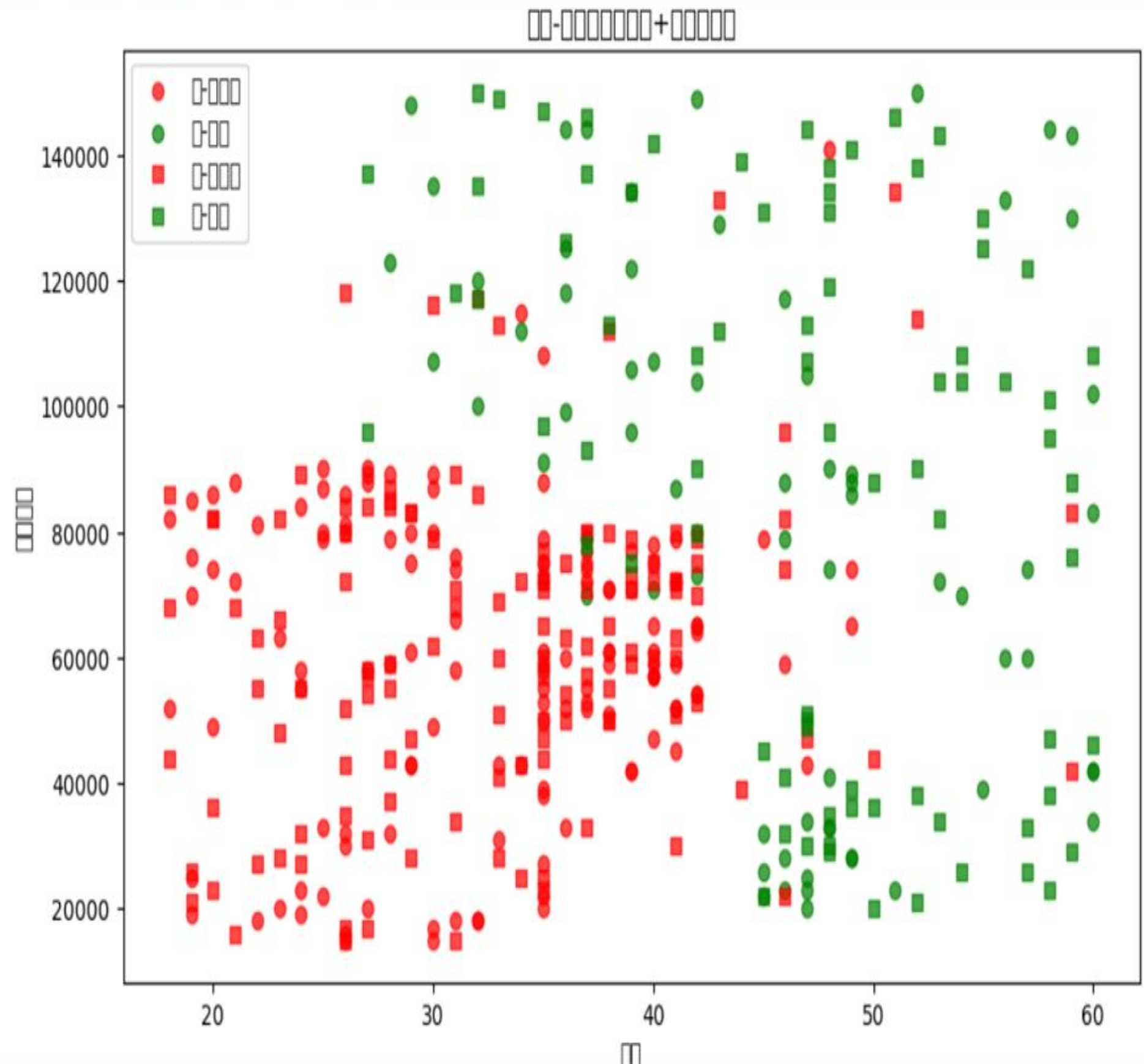
对比训练集和测试集准确率，差距过大且训练集接近100%时，说明出现了**过拟合**。

作业3 —— 可视化图修改

可视化设计

- **横轴**：年龄（Age）
- **纵轴**：预计薪资（EstimatedSalary）
- **颜色**：购买状态（红=未购买 / 绿=购买）
- **形状**：性别（圆=男 / 方=女）

```
plt.figure(figsize=(10,6))
male_0 = df[(df.Gender=='Male') & (df.Purchased==0)]
male_1 = df[(df.Gender=='Male') & (df.Purchased==1)]
female_0 = df[(df.Gender=='Female') & (df.Purchased==0)]
female_1 = df[(df.Gender=='Female') & (df.Purchased==1)]
plt.scatter(male_0.Age, male_0.EstimatedSalary, c='red', marker='o', label='男-未购买',
plt.scatter(male_1.Age, male_1.EstimatedSalary, c='green', marker='o', label='男-购买',
plt.scatter(female_0.Age, female_0.EstimatedSalary, c='red', marker='s', label='女-未购
plt.scatter(female_1.Age, female_1.EstimatedSalary, c='green', marker='s', label='女-购
plt.xlabel('年龄')
plt.ylabel('预计薪资')
plt.title('年龄-薪资分布（性别+购买状态）')
plt.legend()
plt.show()
```



09

第九讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

数据预处理（基础代码）

预处理步骤

- 1 导入库：numpy, matplotlib, pandas
- 2 数据集：Social_Network_Ads.csv
- 3 特征：Age和EstimatedSalary作为X
- 4 标签：Purchased作为y
- 5 划分：test_size=0.2, random_state=0

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 导入数据集
dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
# 使用Age和EstimatedSalary作为特征
X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

| 作业1 —— 加入性别因素，看看Acc的变化

实验设计

对比原始特征（Age+Salary）vs 加入Gender（LabelEncoder编码后）的决策树分类准确率。

特征工程取舍原则

- 并非特征越多越好
- 无关特征可能降低准确率
- 过多无关特征可能导致过拟合

结论

性别对购买行为区分度有限，加入后准确率变化不明显，说明Gender不是**关键预测特征**。

```
# 方法: 使用LabelEncoder将Gender转为数值
from sklearn.preprocessing import LabelEncoder

# 加入Gender特征
X_with_gender = dataset.iloc[:, [1, 2, 3]].values # Gender, Age, EstimatedSalary
y = dataset.iloc[:, 4].values

# 将Gender列进行LabelEncoder编码
le = LabelEncoder()
X_with_gender[:, 0] = le.fit_transform(X_with_gender[:, 0]) # Female=0, Male=1

# 划分训练集和测试集
X_train_g, X_test_g, y_train_g, y_test_g = train_test_split(
    X_with_gender, y, test_size=0.2, random_state=0
)

# 特征缩放
X_train_g = sc.fit_transform(X_train_g)
X_test_g = sc.transform(X_test_g)

# 训练决策树模型
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion='entropy')
clf.fit(X_train_g, y_train_g)

# 预测和评估
y_pred_g = clf.predict(X_test_g)
from sklearn.metrics import confusion_matrix
cm_g = confusion_matrix(y_test_g, y_pred_g)
acc_g = (cm_g[0,0] + cm_g[1,1]) / (cm_g[0,0] + cm_g[0,1] + cm_g[1,0] + cm_g[1,1])
print("加入性别后的准确率:", acc_g)
```

| 作业2 —— 不进行特征缩放，预测准确性有什么变化？

实验设计

对比有/无特征缩放的决策树准确率，验证决策树是否需要特征缩放。

决策树特点

- 基于特征**阈值分裂**
- 与量纲**无关**
- **不需要**特征缩放

对比模型

模型类型	是否需要缩放	原因
决策树/随机森林	不需要	基于阈值分裂
KNN/SVM/Logistic	必须	基于距离或梯度

```
5]: # 不使用特征缩放的版本
X_train_no_scaler, X_test_no_scaler, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 直接训练（不进行StandardScaler）
clf_no_scaler = tree.DecisionTreeClassifier(criterion='entropy')
clf_no_scaler.fit(X_train_no_scaler, y_train)

# 预测和评估
y_pred_no = clf_no_scaler.predict(X_test_no_scaler)
cm_no = confusion_matrix(y_test, y_pred_no)
acc_no = (cm_no[0,0] + cm_no[1,1]) / (cm_no[0,0] + cm_no[0,1] + cm_no[1,0] + cm_no[1,1])
print("不进行特征缩放的准确率:", acc_no)
```

不进行特征缩放的准确率：0.6875
有特征缩放的准确率：0.6875

作业3 —— 比较KNN、Logistic、决策树的表现

模型	准确率	优势	劣势
KNN (k=5)	~63.75%	简单、无参数训练	计算慢、需缩放
Logistic回归	~67.50%	可解释性强	线性边界限制
决策树	~68.75%	无需缩放、可视化强	易过拟合

决策树准确率最高的原因

- 能捕捉非线性关系
- 对特征尺度不敏感
- 通过阈值分裂适应数据分布

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn import tree

# 准备数据 (使用标准化后的数据)
X_train_std, X_test_std, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
sc = StandardScaler()
X_train_std = sc.fit_transform(X_train_std)
X_test_std = sc.transform(X_test_std)

# 1. KNN模型 (k=5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_std, y_train)
y_pred_knn = knn.predict(X_test_std)
acc_knn = accuracy_score(y_test, y_pred_knn)

# 2. Logistic回归模型
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_std, y_train)
y_pred_log = log_reg.predict(X_test_std)
acc_log = accuracy_score(y_test, y_pred_log)

# 3. 决策树模型 (信息增益)
dt = tree.DecisionTreeClassifier(criterion='entropy')
dt.fit(X_train_std, y_train)
y_pred_dt = dt.predict(X_test_std)
acc_dt = accuracy_score(y_test, y_pred_dt)

print("KNN准确率:", acc_knn)
print("Logistic回归准确率:", acc_log)
```

作业4 —— 把决策树输出看看长什么样

决策树配置

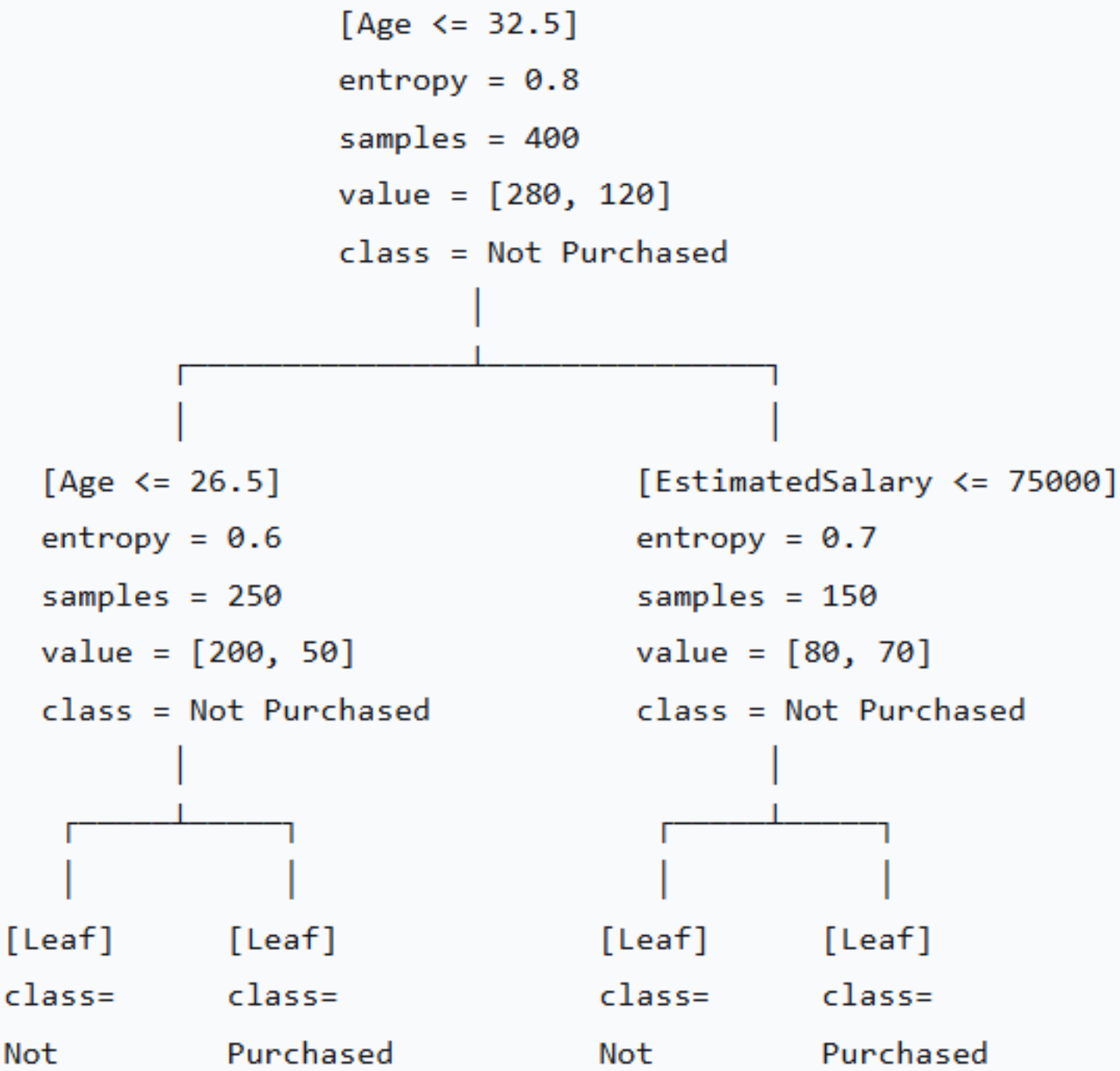
- 模型：DecisionTreeClassifier(criterion='entropy', max_depth=3)
- 准确率约0.9125

混淆矩阵

[[53, 5], [2, 20]] —— 正确分类73个，错误分类7个

节点信息解读

节点字段	含义
entropy	信息熵，越接近0越纯
samples	该节点包含的样本数
value	各类别样本数 [未购买, 购买]
class	该节点的预测类别



10

第十讲作业

姓名：陆晨茜

学号：202321054031

班级：信管2302

| 基础代码（导入库和数据集）

导入库

matplotlib, numpy, sklearn.cluster.KMeans,
sklearn.datasets.load_iris

数据集信息

特征	列索引	含义
sepal length	0	花萼长度
sepal width	1	花萼宽度
petal length	2	花瓣长度
petal width	3	花瓣宽度
数据规模	150×4	150个样本

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
iris = load_iris()
print(iris.data.shape)  # (150, 4)
print(iris.feature_names)
```

作业1 —— 用花萼（第0列和第1列）的数据来聚类

数据提取

- `X_sepal = iris.data[:, :2]`
- 数据形状：150×2（花萼长度+花萼宽度）

聚类设置

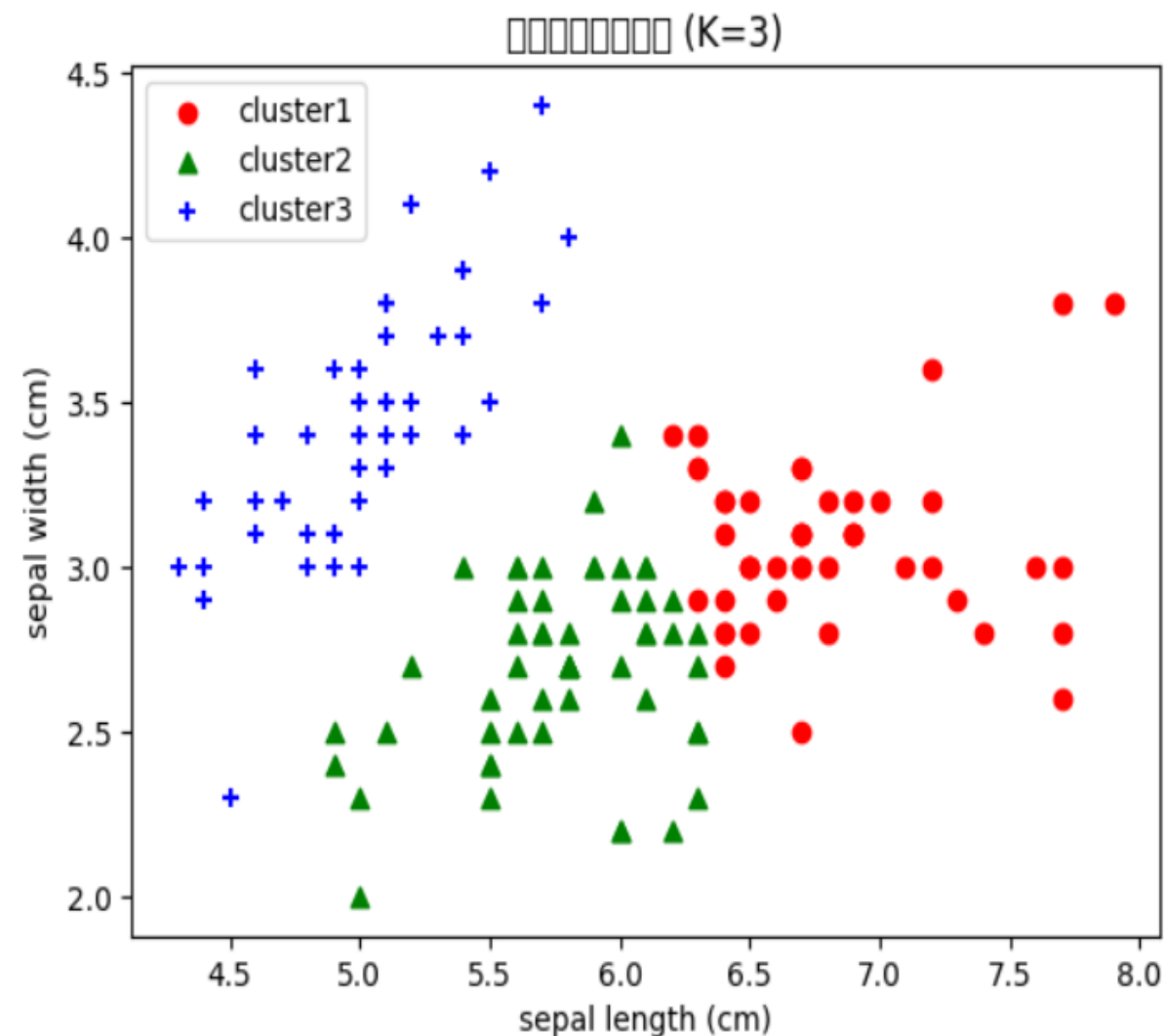
- `KMeans(n_clusters=3)`
- 2维数据可直接可视化

```
# 使用花萼长度（第0列）和花萼宽度（第1列）
X_sepal = iris.data[:, :2] # 150行 × 2列
print("花萼数据形状:", X_sepal.shape)
# 绘制花萼数据散点图
plt.scatter(X_sepal[:, 0], X_sepal[:, 1], c="red")
plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')
plt.title('鸢尾花数据集 - 花萼长度 vs 花萼宽度')
plt.legend(loc=2)
plt.show()
```

```
# 构造聚类器，设置聚类数K=3
estimator_sepal = KMeans(n_clusters=3)
estimator_sepal.fit(X_sepal)

# 获取聚类标签
label_pred_sepal = estimator_sepal.labels_
print("聚类标签:", label_pred_sepal)
# 按聚类标签分组
x0 = X_sepal[label_pred_sepal == 0]
x1 = X_sepal[label_pred_sepal == 1]
x2 = X_sepal[label_pred_sepal == 2]
```

```
# 绘制聚类结果
plt.scatter(x0[:, 0], x0[:, 1], c='red', marker='o', label='cluster1')
plt.scatter(x1[:, 0], x1[:, 1], c='green', marker='^', label='cluster2')
plt.scatter(x2[:, 0], x2[:, 1], c='blue', marker='+', label='cluster3')
plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')
plt.title('花萼数据聚类结果 (K=3)')
plt.legend(loc=2)
plt.show()
```



作业2 —— 用所有数据来聚类（所有4列）

数据提取

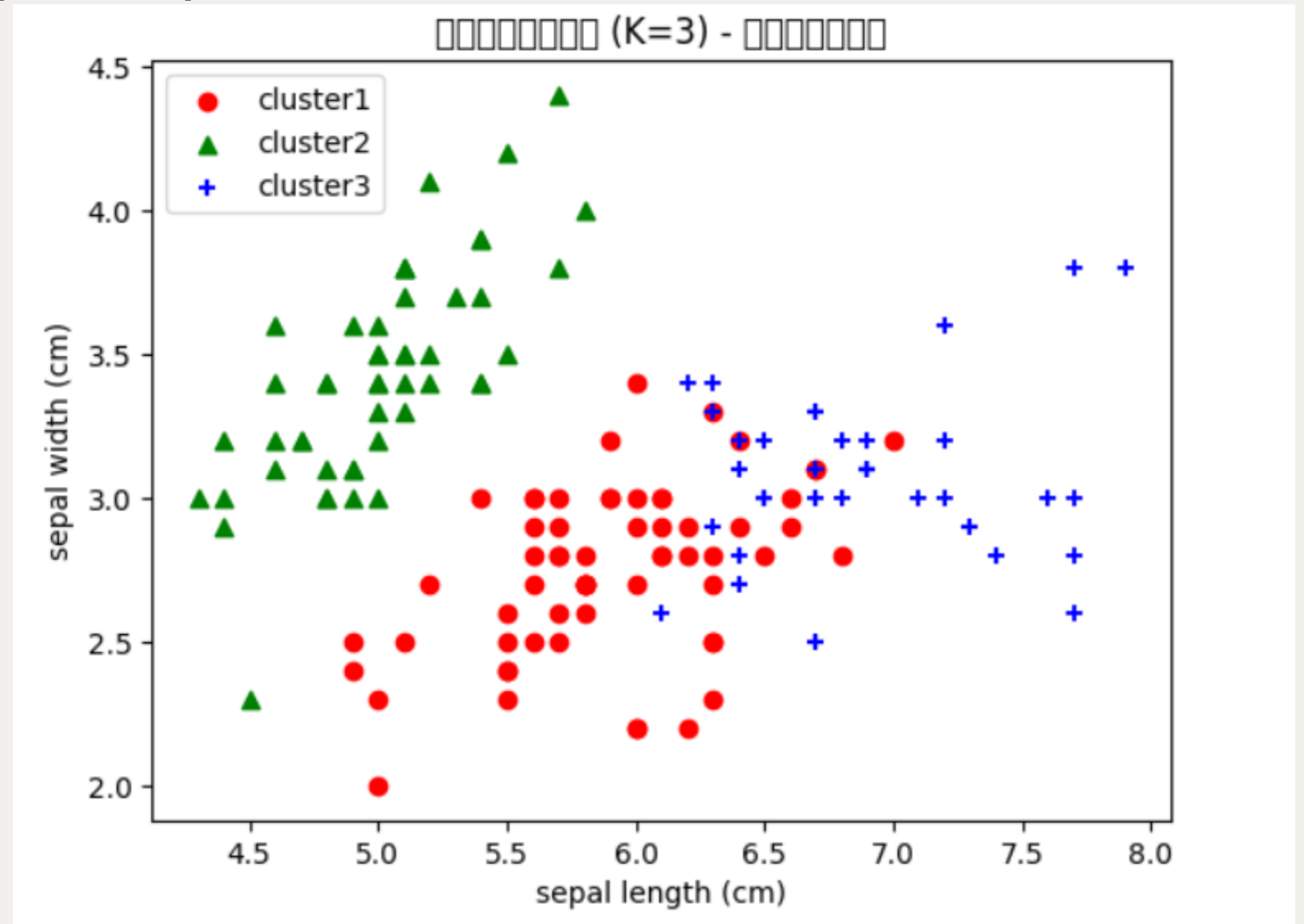
- $X_{all} = iris.data$ （全部4个特征）
- 数据形状：150×4

聚类设置

- $K=3$ ，4维数据
- 需选择维度展示（用花萼维度作为坐标轴）

与作业1对比

- 全部4个特征信息**更全面**
- 聚类效果更准确
- 高维数据可视化需降维或选维度



作业3 —— 把K改成2，试一试

实验设计

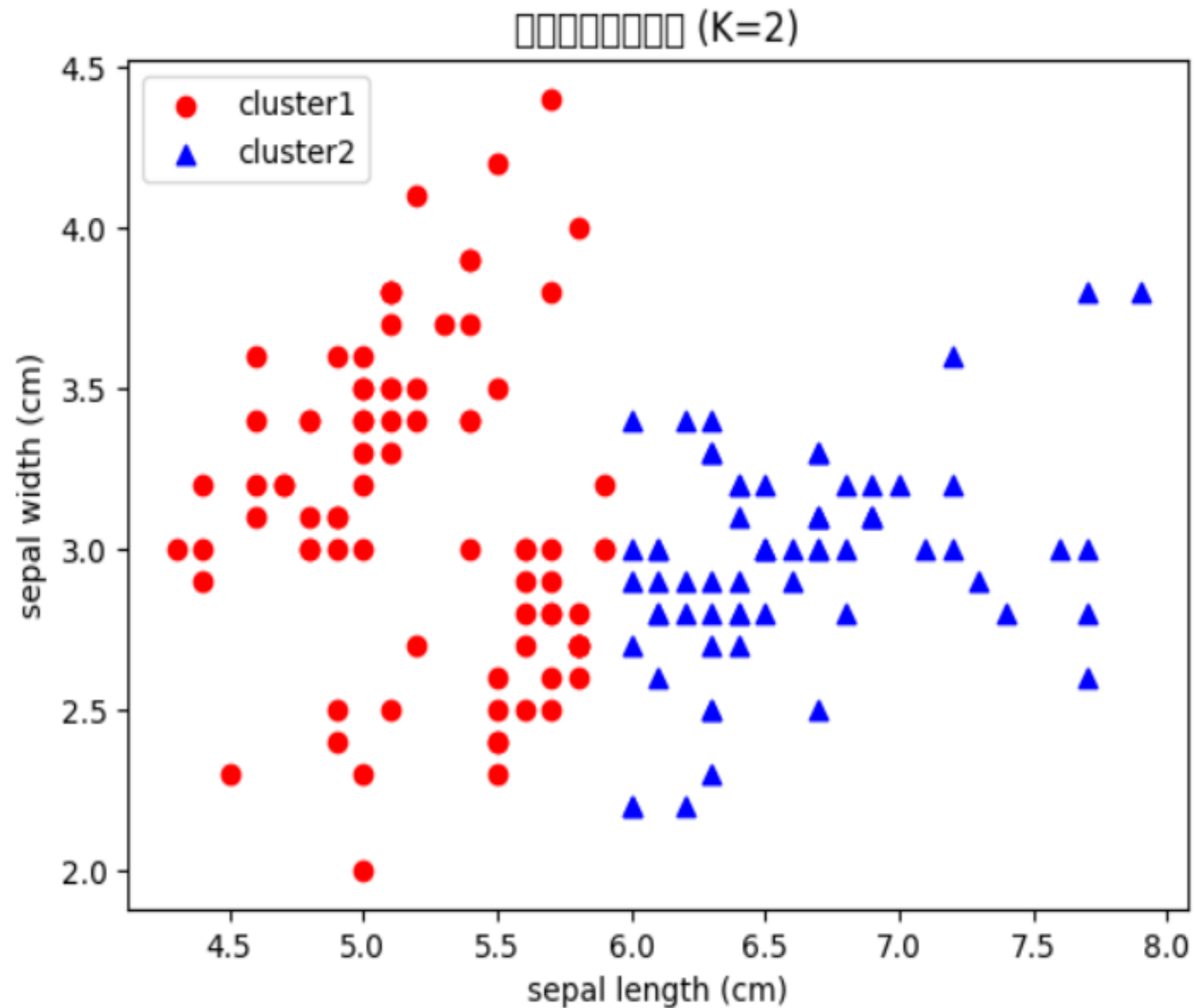
- 花萼数据K=2 : `X_sepal + KMeans(n_clusters=2)`
- 全部数据K=2 : `X_all + KMeans(n_clusters=2)`
- 对比K=2与K=3的聚类结果差异

K=2 vs K=3 对比

K=2：分两大类，边界较粗，部分样本被强制合并

K=3：划分更细，能区分三个花种，与真实类别基本吻合

结论：K值影响聚类粒度，需根据数据特点合理选择



| 作业4 —— 聚类中心获取与可视化

聚类中心获取

- 通过 `estimator.cluster_centers_` 属性获取
- 形状为 (K, 特征数), 即3×4的矩阵

聚类中心可视化

- **柱状图**: 展示各簇在4个特征上的中心值
- **散点图**: 标注中心点位置, 对比簇内样本分布
- 聚类中心代表每个簇的**典型特征**

轮廓系数评估

- `silhouette_score` 评估聚类效果
- 取值范围 [-1, 1], 越接近1越好
- 可量化对比不同K值和不同特征的聚类质量

```
[12]: # 使用全部数据, K=3
estimator = KMeans(n_clusters=3, n_init=10, random_state=42)
estimator.fit(X_all)

# 获取聚类中心
cluster_centers = estimator.cluster_centers_
print("聚类中心 (4维):")
print(cluster_centers)
print("\n聚类中心形状:", cluster_centers.shape) # (3, 4)

# 获取每个特征的聚类中心值
feature_names = ['sepal length', 'sepal width', 'petal length', 'petal width']
for i, center in enumerate(cluster_centers):
    print(f"\n簇 {i+1} 的中心:")
    for j, name in enumerate(feature_names):
        print(f"    {name}: {center[j]:.2f}")
```

聚类中心 (4维):

```
[[5.9016129  2.7483871  4.39354839 1.43387097]
 [5.006       3.428       1.462       0.246       ]
 [6.85       3.07368421 5.74210526 2.07105263]]
```


作业4 —— 聚类中心获取与可视化

```
3]: # 方法2: 在花萼维度上展示聚类中心
plt.figure(figsize=(10, 6))

# 绘制所有数据点
plt.scatter(X_all[:, 0], X_all[:, 1], c=estimator.labels_,
            cmap='viridis', alpha=0.6, label='Data points')

# 绘制聚类中心 (使用花萼维度)
plt.scatter(cluster_centers[:, 0], cluster_centers[
            c='red', marker='X', s=200, edgecolors=
            linewidths=2, label='Cluster Centers')

plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')
plt.title('K-means Clustering Results with Cluster Centers')
plt.legend()
plt.show()
```

